



安徽大学
人工智能学院
School of Artificial Intelligence
Anhui University

《智能机器人系统综合训练》课程报告

学 院 人工智能

专 业 机器人工程

学生姓名学号 应宇杰 WA2224137

指导老师 常灯祥

课程编号 _____

课程学分 _____

提交日期 2025.12.30

项目仓库 <https://github.com/vyjgg/RobotPracticalTrainingAhu.git>

目录

一、 实验目的和要求	3
二、 实验原理.....	3
三、 实验原始纪录	7
四、 实验结果及分析	12
五、 实验总结与思考	15

一、 实验目的和要求

（一）实验目的

掌握路径规划核心算法的原理与实现深入理解 A * 算法的全局路径搜索机制（包括节点代价计算、启发式函数设计、开放集 / 闭合集管理）和动态窗口法（DWA）的局部避障逻辑（速度窗口采样、轨迹预测与代价评估），明确两种算法在机器人导航中的互补作用（全局路径提供宏观导向，局部规划处理实时避障）。

（二）实验要求

通过 python 构建一个小工程用于实现 A_star 和 DWA 算法的实现，并且同时实现局部路径规划器和全局路径规划器的联合，对比在静态和动态地图上的算法的实现流程和性能。

二、 实验原理

（一）A_star 全局路径规划算法

1. 核心定义与核心公式

A*算法是一种基于启发式搜索的全局最优路径规划算法，其核心思想是通过预估代价引导搜索方向，在保证路径最优性的前提下提升搜索效率，适用于静态环境下的全局路径规划。

A*算法的核心是对每个搜索节点计算总代价 $f(n)$ ，由实际代价 $g(n)$ 和启发式预估代价 $h(n)$ 组成：

$$f(n) = g(n) + h(n) \quad 1$$

其中 $g(n)$ 表示从起点 S 到当前节点 n 的实际移动代价， $h(n)$ 表示从当前节点 n 到目标点 G 的预估代价，本实验采用欧氏距离作为启发

函数，则有：

$$h(n) = (x_n - x_G)^2 + (y_n - y_G)^2 \quad 2$$

其中 (x_n, y_n) 为当前节点网格坐标， (x_G, y_G) 为目标点网格坐标。

2. 算法流程

(1)预处理阶段：输入障碍物坐标、机器人半径、网格分辨率，计算障碍物网格地图，确定地图最小和最大坐标，完成实际坐标与网格坐标的转换。

(2)初始化阶段：将起点转换为网格节点，加入开放集和优先队列；初始化闭合集为空。

(3)核心搜索阶段：

- a. 从优先队列中弹出总代价 $f(n)$ 最小的节点，作为当前节点。
- b. 若当前节点为目标节点，搜索结束，进入路径回溯阶段。
- c. 将当前节点从开放集移至闭合集，避免重复探索。
- d. 根据 8 方向移动模型，生成所有邻接节点，计算邻接节点的实际代价 $g(n)$ 和总代价 $f(n)$ 。
- e. 验证邻接节点合法性（是否在地图范围内、是否碰撞障碍物）。
- f. 若邻接节点不在开放集，直接加入开放集和优先队列；若已在开放集，对比新老路径的，若新路径代价更低，则更新节点代价和父节点。

(4)路径回溯阶段：从目标节点出发，通过父节点索引反向追溯至起点，再将路径反转，得到从起点到目标点的全局最优路径。

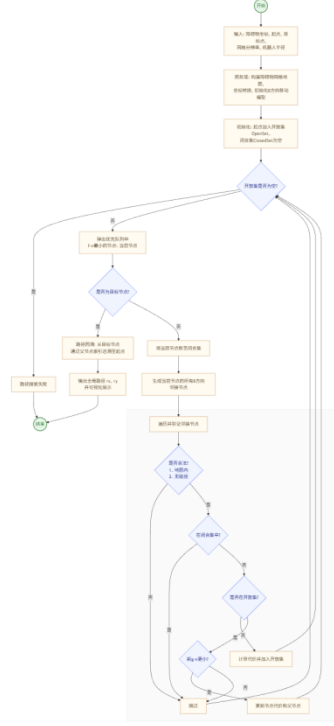


图 2.1 A*算法流程图

(二) DWA 局部路径规划算法

1. 核心定义与核心公式

DWA（动态窗口法）是一种基于速度采样的局部路径规划算法，其核心思想是在机器人动力学约束允许的速度窗口内，采样多组控制量（线速度 v 、角速度 ω ），预测每条控制量对应的轨迹，通过代价函数评估轨迹优劣，选择最优控制量，实现实时避障和轨迹跟踪。

(1)动态窗口计算公式:

$$V_{dw} = V_a \cap V_v \cap V_o \quad 3$$

动态窗口 V_{dw} 是由速度约束 V_v 、加速度约束 V_a 和障碍物约束 V_o 三个约束交集构成，其中：

$$V_v = \{(v, \omega) \mid v_{\min} \leq v \leq v_{\max}, \omega_{\min} \leq \omega \leq \omega_{\max}\} \quad 4$$

$$V_a = \{(v, \omega) \mid v \in [v_{curr} - a_v \Delta t, v_{curr} + a_v \Delta t], \omega \in [\omega_{curr} - a_\omega \Delta t, \omega_{curr} + a_\omega \Delta t]\} \quad 5$$

V_o 使用停止距离 $d_{stop} = \frac{v^2}{2a}$ 来描述。

(2)轨迹预测公式

$$\begin{cases} x_{t+1} = x_t + v \cdot \Delta t \cdot \cos \theta_t \\ y_{t+1} = y_t + v \cdot \Delta t \cdot \sin \theta_t \\ \theta_{t+1} = \theta_t + \omega \cdot \Delta t \end{cases} \quad 5$$

其中 (x_t, y_t, θ_t) 为机器人在 t 时刻的位置和航向角， (v, ω) 为采样的控制量。

(3)代价函数公式

$$J(v, \omega) = w_{goal} \cdot J_{goal}(v, \omega) + w_{obs} \cdot J_{obs}(v, \omega) + w_{speed} \cdot J_{speed}(v, \omega) \quad 6$$

2. 算法流程

1. 参数初始化阶段：加载 DWA 配置参数（速度范围、加速度限制、采样时间、预测时间、代价权重等），获取机器人当前状态（位置、航向角、速度）。

2. 动态窗口计算阶段：根据速度约束、加速度约束、障碍物约束，计算可行的速度窗口 V_{dw} 。

3. 轨迹采样与预测阶段：在动态窗口 V_{dw} 内，均匀采样多组控制量 (v_i, ω_i) ，通过机器人运动学模型，预测每组控制量对应的局部轨迹。

4. 轨迹代价评估阶段：对每条预测轨迹，分别计算目标方向代价、障碍物代价、速度代价，按加权和公式计算总代价 $J(v_i, \omega_i)$ 。

5. 最优控制量选择阶段：遍历所有采样轨迹，选择总代价最小的轨迹对应的控制量 (v_{opt}, ω_{opt}) 。

6. 机器人状态更新阶段：将最优控制量作用于机器人，更新机器人的位置、航向角和速度，完成一次局部规划。
7. 循环执行：重复步骤 2-6，直至机器人到达目标点。



图 2.2 DWA 算法流程图

（三）全局+局部路径规划算法

在实际完成路径规划任务时，通常先使用全局规划器得到一个大体的路径序列点，再使用实时的局部路径规划器去动态的避开障碍物，其中局部规划器在每一个小的阶段的目标点是由全局路径给出的。

三、 实验原始纪录

本次实验是基于 python 的工程，分为三个代码文件来编写，main.py 主要是存放生成地图函数、调用全局规划器和局部规划器和绘制全局和实时规划的路径信息。a_star.py 主要是定义了一个 A*算

法实现的对象。dwa.py 主要是定义了实现 dwa 局部路径规划器的函数。在本实验报告中只添加主函数接口的这部分代码：

```
# --- 主程序 ---
def main():
    print("-----")
    print("    Hybrid A* + DWA 路径规划仿真    ")
    print("-----")
    print("请选择地图类型:")
    print("1. 简单迷宫")
    print("2. 复杂迷宫")
    print("3. 动态复杂迷宫 (A* 路径上将出现红色障碍)")

    choice = input("请输入选项 (1, 2 或 3): ").strip()

    if choice == '2':
        static_ox, static_oy, sx, sy, gx, gy = design_map_complex()
        is_dynamic_path_obs = False
    elif choice == '3':
        static_ox, static_oy, sx, sy, gx, gy = design_map_dynamic_complex()
        is_dynamic_path_obs = True
    else:
        static_ox, static_oy, sx, sy, gx, gy = design_map_simple()
        is_dynamic_path_obs = False

    #A* 全局规划阶段
    ox_a_star = static_ox
    oy_a_star = static_oy
    if show_animation:
        plt.figure(figsize=(8, 8))
        plt.plot(ox_a_star, oy_a_star, ".k", label="静态障碍物")
        plt.plot(sx, sy, "og", label="起点")
        plt.plot(gx, gy, "xb", label="终点")
        # 这里先不显示 A* 路径, 因为它可能被动态障碍覆盖, 待动态障碍放置后再显示
        # 如果是动态迷宫, 动态障碍物还没放置, 所以这里也不显示其图例
        plt.legend(loc='upper right') # 提前显示起点终点静态障碍
        plt.grid(True)
        plt.axis("equal")
        plt.title("A* 全局路径规划 (基于静态地图)")

    print("1. 正在运行 A* 全局规划...")
    a_star_planner = a_star.AStarPlanner(ox_a_star, oy_a_star, GRID_SIZE, ROBOT_RADIUS)
```



```

rx, ry = a_star_planner.planning(sx, sy, gx, gy, show_process=show_animation)

rx = rx[::-1]
ry = ry[::-1]

if not rx:
    print("A* 无法找到路径！")
    if show_animation: plt.show()
    return

#添加动态障碍物
dynamic_ox, dynamic_oy = [], []
if is_dynamic_path_obs:
    dynamic_ox, dynamic_oy = place_dynamic_obs_on_path(rx, ry)
    # 此时 A* 规划已完成, 绘制完整的 A* 路径和动态障碍物
    if show_animation:
        # 清除之前的图例, 重新绘制以包含 A* 路径和动态障碍物
        plt.cla()
        plt.plot(ox_a_star, oy_a_star, ".k", label="静态障碍物")
        plt.plot(sx, sy, "og", label="起点")
        plt.plot(gx, gy, "xb", label="终点")
        plt.plot(dynamic_ox, dynamic_oy, "or", markersize=5, label="动态障碍物 (A*未知)") #
        红色圆点
        plt.plot(rx, ry, "-r", linewidth=2, label="A*全局路径") # 红色线表示全局路径
        plt.legend(loc='upper right')
        plt.title("A* 全局路径及动态障碍物")
        plt.pause(0.5) # 暂停一下, 让用户看清
        print("2. 动态障碍物已添加, DWA 将开始追踪并避障。")
else:
    # 如果没有动态障碍物, 也显示 A* 路径
    if show_animation:
        plt.plot(rx, ry, "-r", linewidth=2, label="A*全局路径")
        plt.legend(loc='upper right')
        plt.title("A* 全局路径已规划")
        plt.pause(0.5)
        print("2. A* 全局路径已规划, DWA 将开始追踪。")

#DWA 局部规划阶段
print("3. 开始 DWA 轨迹追踪...")

x = np.array([sx, sy, math.pi / 8.0, 0.0, 0.0])
goal = np.array([gx, gy])

static_ob_np = np.array([static_ox, static_oy]).T
dynamic_ob_np = np.array([dynamic_ox, dynamic_oy]).T if dynamic_ox else np.empty((0, 2))

```

```

config = dwa.Config()
config.robot_radius = ROBOT_RADIUS
trajectory = np.array(x)
target_ind = 0
stuck_counter = 0

while True:
    combined_ob = np.vstack((static_ob_np, dynamic_ob_np))

    dist_to_goal = math.hypot(x[0] - gx, x[1] - gy)
    min_dist = float("inf")
    for i in range(target_ind, len(rx)):
        d = math.hypot(rx[i] - x[0], ry[i] - x[1])
        if d < min_dist:
            min_dist = d
            target_ind = i
        else:
            break

    lookahead_offset = 2
    local_goal_found = False
    while not local_goal_found:
        current_lookahead_idx = min(target_ind + lookahead_offset, len(rx) - 1)
        temp_goal = np.array([rx[current_lookahead_idx], ry[current_lookahead_idx]])
        if current_lookahead_idx == len(rx) - 1:
            local_goal = temp_goal
            local_goal_found = True
        elif not is_path_point_obstructed(temp_goal[0], temp_goal[1], combined_ob,
config.robot_radius):
            local_goal = temp_goal
            local_goal_found = True
        else:
            lookahead_offset += 1
            if lookahead_offset > 50:
                local_goal = temp_goal
                local_goal_found = True
                # print("Warning: Lookahead search reached limit.") # 可以选择打印警告

    u, predicted_traj, all_candidates = dwa.dwa_control(x, config, local_goal,
combined_ob)

    if abs(u[0]) < STUCK_LINEAR_VELOCITY_THRESHOLD and abs(u[1]) < 0.1:
        stuck_counter += 1
    else:
        stuck_counter = 0

```

```

if stuck_counter >= STUCK_COUNT_MAX:

    print("\n!!! --- DWA 陷入局部困境，触发全局路径重规划 --- !!!")

    # 将动态障碍物变为静态障碍物，并清空动态列表
    static_ox.extend(dynamic_ox)
    static_oy.extend(dynamic_oy)
    dynamic_ox, dynamic_oy = [], []

    print("    -> 重新计算 A* 路径...")
    a_star_planner_new = a_star.AStarPlanner(static_ox, static_oy, GRID_SIZE,
ROBOT_RADIUS)
    rx_new, ry_new = a_star_planner_new.planning(x[0], x[1], gx, gy,
show_process=False)

    if not rx_new:
        print("    -> 警告：重规划失败！目标点被完全封锁。")
        break

    rx, ry = rx_new[::-1], ry_new[::-1]
    target_ind = 0
    stuck_counter = 0
    print("    -> 全局路径更新完成，继续 DWA 追踪。")

    static_ob_np = np.array([static_ox, static_oy]).T
    dynamic_ob_np = np.empty((0, 2)) # 确保动态障碍物在 DWA 输入中也被清空
    x = dwa.motion(x, u, config.dt)
    trajectory = np.vstack((trajectory, x))
    if dist_to_goal <= config.robot_radius:
        print("到达目标点!")
        break

```

```

if show_animation:
    plt.cla()

    # 1. 绘制静态障碍物 (黑色点)
    plt.plot(static_ox, static_oy, ".k", label="静态障碍物")

    # 2. 绘制动态障碍物 (红色圆点)
    if dynamic_ox:
        plt.plot(dynamic_ox, dynamic_oy, "or", markersize=5, label="动态障碍物")

    # 3. 绘制起点、终点和局部目标
    plt.plot(sx, sy, "og", label="起点")
    plt.plot(gx, gy, "xb", label="终点")

```

```

plt.plot(local_goal[0], local_goal[1], "xg", markersize=8, label="局部目标") # 局部
目标, 加大标记

# 4. 绘制 A* 全局路径

plt.plot(rx, ry, "-r", linewidth=2, label="A*全局路径")

# 5. 绘制 DWA 采样过程 (绿色半透明细线)
for i in range(0, len(all_candidates), 5):
    cand = all_candidates[i]
    plt.plot(cand[:, 0], cand[:, 1], "-", color=[0, 1, 0, 0.15])
plt.plot([], [], "-", color=[0, 1, 0, 0.15], label="DWA 采样路径") # 为采样路径添加一
个图例条目

# 6. 绘制 DWA 选定的最优预测轨迹 (蓝色粗线)
plt.plot(predicted_traj[:, 0], predicted_traj[:, 1], "-b", linewidth=2, label="DWA
最优轨迹")

# 7. 绘制机器人历史轨迹 (黑色实线)
plt.plot(trajectory[:, 0], trajectory[:, 1], "-k", label="机器人历史轨迹")

# 8. 绘制当前机器人位置和朝向 (蓝色圆和线)
draw_robot(x[0], x[1], x[2], config, label="机器人") # 给机器人添加图例
plt.title("DWA 局部规划 (停滞计数: {})".format(stuck_counter))
plt.axis("equal")
plt.grid(True)

plt.legend(loc='upper right', prop={'size': 9}) # 调整图例字体大小和位置
plt.pause(0.001)

print("Done")

if show_animation:
    plt.show()

```

四、 实验结果及分析

本次实验主要设计了三种地图，分别是简单地图、复杂地图（静态障碍物）和复杂地图（含有动态障碍物），在每一次路径规划中，都会先进行全局路径规划获得一条全局路径（使用红色粗线标注出来），在实时的局部路径规划中，使用绿色细线表示每一帧中 dwa 考虑的所有的可能的路径，绿色箭头是局部规划器当前的目标点，最后蓝色粗线为最优路径。在局部路径规划的主循环中，为了防止机器人陷入局部最优点，通过检测机器人的速度和停滞次数，来判断是否需要重新进行全局路径规划。

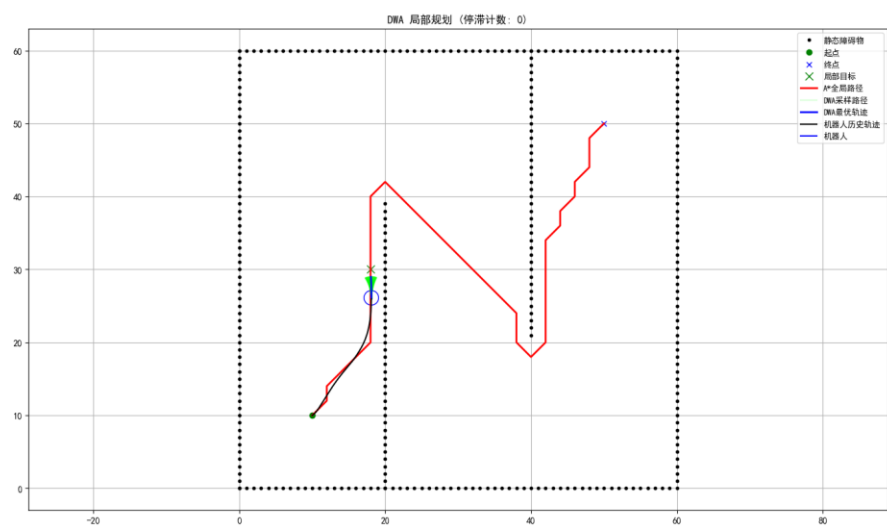


图 4.1 简单地图的局部路径规划

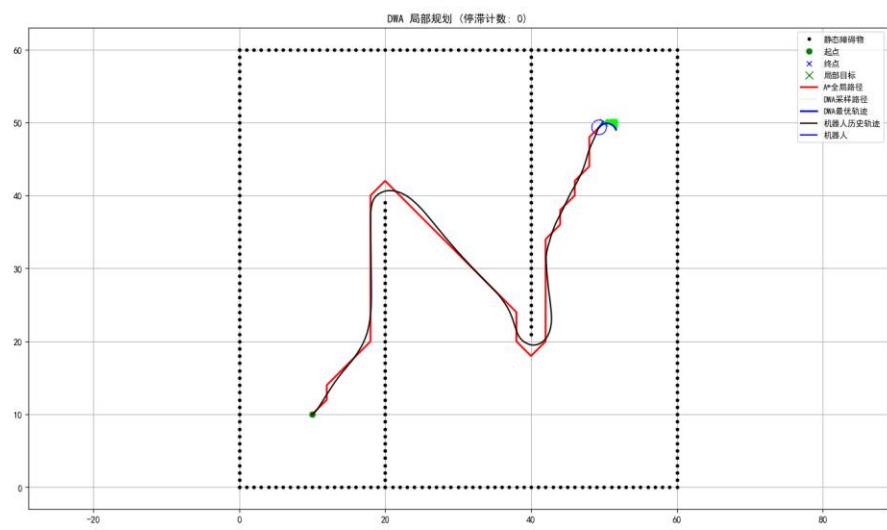


图 4.2 简单地图最终路径

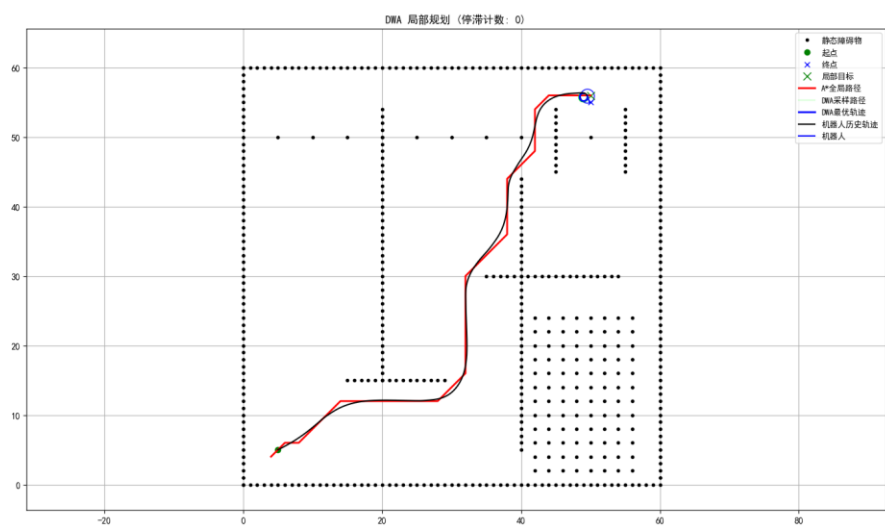


图 4.3 复杂地图最终路径

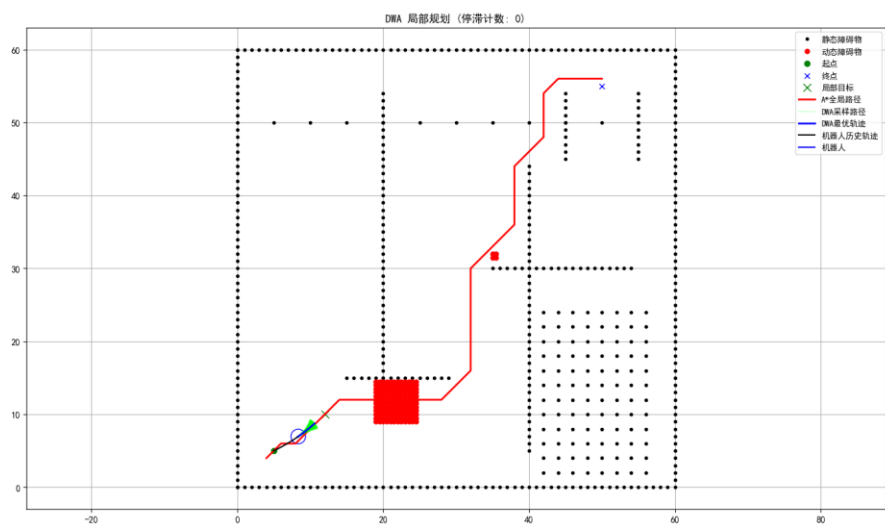


图 4.4 复杂地图的局部路径规划

在图 4.4 中，红色点是动态障碍物，即在全局路径规划中，并没有发现它们。

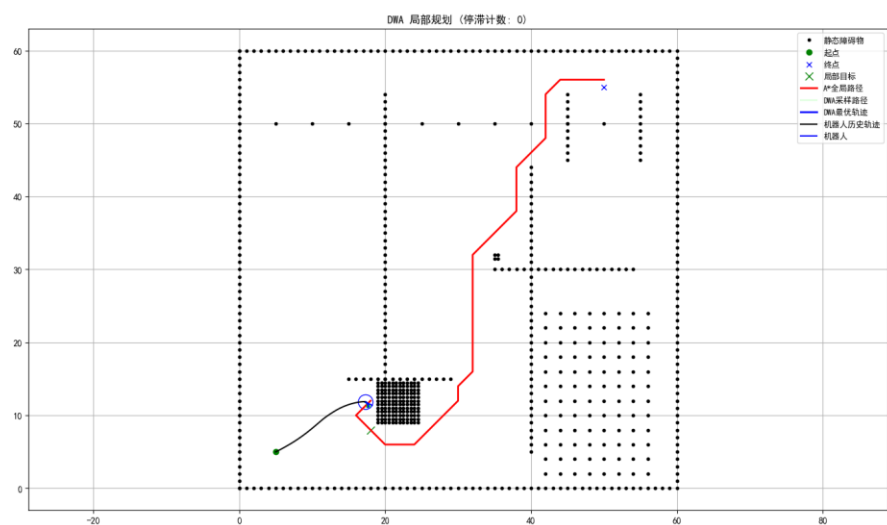


图 4.5 在陷入局部最优时重新进行全局规划

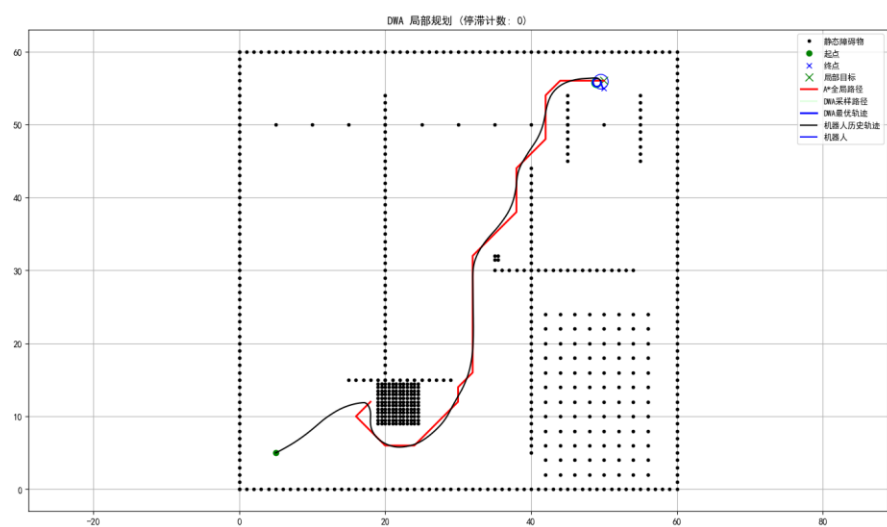


图 4.6 复杂地图最终路径

五、 实验总结与思考

本次实验围绕 A 全局路径规划算法与 DWA 局部避障算法的联合应用展开，成功通过 Python 实现了两者的融合框架，并在简单迷宫、复杂静态迷宫及动态障碍迷宫三种场景下完成了验证。实验中，A 算

法凭借启发式搜索高效生成全局最优路径，为机器人提供宏观导航方向，DWA 算法则通过速度窗口采样、轨迹预测与代价评估，实现了实时避障与轨迹跟踪，两者形成良好互补。同时，通过设计停滞检测机制，当机器人陷入局部困境时能触发全局路径重规划，提升了系统的鲁棒性。实验结果表明，该联合框架在静态环境中能精准规划最优路径，在动态环境中可有效应对突发障碍，不过在复杂障碍密集场景下，算法的实时性仍有优化空间，后续可通过改进启发函数、优化采样策略等方式进一步提升性能。